

Where the ledger can still be erased

A classification of every code path that physically removes posted accounting rows, into the four operations professional practice distinguishes: edit, reverse, void, delete. Prepared as the sizing step before any reversal service is written.

DATE	BUILD	SITES CLASSIFIED	LEDGER REACHABLE
13 September 2026	master @ eeb49e0e	19	5,194 of 5,213 posted
PERIODS CLOSED			
0 of 6			

1. Summary

The 6 September report counted 13 delete paths. A full sweep of the Application layer for any statement that removes `AcctgTrans` , `AcctgTransEntry` , `FinAccountTrans` or `Payment` rows found six more, bringing the register to 19. Each was read in full, its precondition and its blast radius recorded, and its exposure measured against the live database.

Classification is by what the artifact is allowed to become, not by what the code does today. Seven sites must become **reversals**, six must become **voids**, three keep a physical delete behind a guard, two are edit paths that must refuse once posted, and one is dead code to retire. No site is correct as written. The one handler that already gets it right, `DeleteParty` , refuses when ledger rows exist and is the pattern every guarded delete copies.

7

REVERSE

Posted and legitimately re-openable. Contra transaction, original stays.

6

VOID

Artifact should never have existed. Status → cancelled, plus reversal. Row never removed.

3

DELETE, GUARDED

Physical delete stays, but only while nothing is posted and nothing refers to it.

2

EDIT ONLY UNPOSTED

In-place change of ledger rows. Refuse once posted.

1

RETIRE

Unreachable code that still carries a delete.

2. The rule applied

The auditor's instruction, "deletes should be marked as deleted, not removed", is applied with the boundary agreed in the September report: it governs anything that has ever touched the general ledger, plus anything referenced by something that has. Drafts that never posted may still be removed outright. Four operations follow from that, and every site below is assigned exactly one, sometimes with a second for its draft state.

Operation	Precondition	What happens to the ledger	What happens to the artifact
EDIT	Nothing posted yet	Nothing	Changed in place
REVERSE	Posted, period open, artifact will be corrected and re-posted	New contra transaction, opposite signs, dated in the open period, linked to the original. Both visible.	Returns to its editable status
VOID	Posted, artifact should never have existed	Same reversal as above	Status → *_CANCELLED / PMNT_VOID . Row stays, excluded from operational lists.
DELETE	Never posted, nothing references it	Nothing to do	Physically removed. Refused otherwise.

3. Exposure by anchor

Every transaction carries the id of the artifact that created it. Counting posted transactions by that anchor shows which sites hold the most evidence. Figures are from the local `erp_contracts` database on 13 September.

Anchor on ACCTG_TRANS	Rows	Posted	Sites that delete them
Payment	3,351	3,351	ResetPayment, DeletePayment, DeleteEmployeeAdvance, payroll wipes, certificate resets, sales-request purges
Work effort (certificate)	1,767	1,767	ResetProjectCertificate, ResetMultiPaymentCertificate, DeleteMultiPaymentCertificate
Sales request	952	952	DeleteSalesRequest, ResetSalesRequest
Invoice	550	550	ResetInvoice, payroll wipes, ResetProjectCertificate
Shipment or receipt	242	242	ResetProjectCertificate
Manual journal (no anchor)	242	223	DeleteAcctgTrans, DeleteAcctgTransEntry
All	5,213	5,194	Only 19 transactions in the whole ledger are unposted

Anchors overlap, so the rows do not sum to the total. The point is the shape: payments and certificates hold four-fifths of the evidence.

4. The register

Each site records the entry point, what the code refuses today, what it removes, the live exposure, the classification, and the change that classification implies. A **flagged** line is a defect independent of the auditor's finding.

Payments

1 ResetPayment **REVERSE**

ENTRY	POST /payments/reset/{id} · Payment form action "إعادة تعيين الدفعة", gated by Can perform="Reset_Payment"
REFUSES TODAY	Status <code>PMNT_VOIDED</code> , <code>PMNT_CANCELLED</code> , <code>PMNT_CONFIRMED</code> . PMNT_VOIDED is not a seeded status; the seeded id is PMNT_VOID, so a voided payment is not actually refused.
REMOVES	Payment applications, then every transaction and entry keyed by the payment id. Status back to <code>PMNT_NOT_PAID</code> , override account cleared.
LEAVES BEHIND	The FinAccountTrans row is not touched, which is correct: it is created with the payment at draft time and belongs to the payment, not to its posting. Invoices the payment had settled stay <code>INVOICE_PAID</code> . Transactions anchored only by fin-account-trans id are missed.
EXPOSURE	2,978 sent or received payments can be reset.
BECOMES	Posted → reversal linked to the original, applications removed and their invoices returned to <code>INVOICE_READY</code> , status <code>PMNT_NOT_PAID</code> . The bank row stays with the payment. Draft → nothing to do. Refused in a closed period.

2 DeletePayment **VOID** **DELETE WHILE DRAFT**

ENTRY	DELETE /payments/{id}
REFUSES TODAY	Linked to a sales request, an employee advance, a project certificate, or applied to an invoice. No status guard: a PMNT_SENT payment with bank and ledger rows is deleted whole.
REMOVES	Transactions and entries by payment id, fin-account rows, then the payment.
BECOMES	Draft with no ledger or bank rows → physical delete, guarded the DeleteParty way. Anything else → <code>PMNT_VOID</code> plus reversal, fin-account row to <code>FINACT_TRNS_CANCELED</code> . The OFBiz port <code>PaymentHelperService.VoidPayment</code> already does most of this and is the primitive to promote (section 6).

3 DeleteEmployeeAdvance **VOID** **DELETE WHILE DRAFT**

ENTRY	DELETE /humanresources/{id}?dropPayment=
REFUSES TODAY	Advance already on a payroll invoice, or any deduction started.
REMOVES	Schedules, the advance, and, when its payment is draft or <code>DropPayment</code> is passed, the payment with its transactions and bank row. The DropPayment override deletes a disbursed payment.
EXPOSURE	288 advances, 287 of them with a payment beyond draft.
BECOMES	Payment disbursed → advance <code>ADVANCE_CANCELLED</code> and the payment voided through the same primitive as site 2. Payment still draft → delete both. The override goes.

4 UpdatePayment → DeletePostdatedChequeAccountingTransaction

REVERSE AND RE-POST

ENTRY	PUT /payments (edit form) · GeneralLedgerService.DeletePostdatedChequeAccountingTransaction
TRIGGER	Editing a cheque's amount, account or details deletes the <code>CHECK_ISSUED</code> transaction and creates a fresh one; clearing the bank account removes the fin-account row. The method's own comment names reversal as "Option B, safer audit trail".
BECOMES	Reverse the existing transaction, then post the new one, so the edit is visible as three linked rows. If the auditor accepts the pragmatic pre-send exception from the September report, this path can stay as is while the payment is <code>PMNT_NOT_PAID</code> and the period is open. Needs the written agreement either way.

5 CompleteSalesOrderPayments · UpdateOrApproveOrderPayments

DELETE, GUARDED

ENTRY	Sales-order payment grid, IsPaymentDeleted flag on a row
REMOVES	The payment, its order payment preference and order status rows. Does not look at the ledger at all.
EXPOSURE	Low for this client: the sales-order flow is barely used, and foreign keys would stop the delete if a transaction existed. It is listed because the flag is an unguarded delete of a financial artifact.
BECOMES	Keep the delete, add the DeleteParty refusal when any transaction or fin-account row references the payment.

Payroll

6 ResetInvoice

REVERSE

ENTRY	POST /invoices/reset/{id} · Invoice display form and accounting menu
REFUSES TODAY	Anything but <code>INVOICE_READY</code> .
REMOVES	Status history other than in-process, all transactions by invoice id, and every payment application on the invoice. Applications are deleted without touching the payments on the other side, so a sent payment silently becomes unapplied. The status history is itself audit evidence and should not be erased.
EXPOSURE	550 invoice-anchored posted transactions. All 158 payroll invoices are posted, because the batch sets them <code>INVOICE_READY</code> at generation.
BECOMES	Refuse when any application points at a payment beyond draft. Otherwise reverse the accrual, set <code>INVOICE_IN_PROCESS</code> , keep the status history. Under the pragmatic policy (before payment sent, period open) unpost-edit-repost may be allowed instead; that is the one decision to get from the auditor in writing.

7 DeletePayrollInvoices DELETE, GUARDED

ENTRY	POST /invoices/deletePayrollInvoices · Payroll run screen
REFUSES TODAY	Nothing.
REMOVES	The whole month for the organisation: invoices, items, roles, attributes, status history and their transactions; the aggregate payments addressed to the staff party with their transactions, bank rows and group memberships; advances and schedules put back to scheduled.
BECOMES	The re-run window is legitimate, so the delete stays, but only while every payroll payment of the month is <code>PMNT_NOT_PAID</code> and the period is open. Once anything has been sent, refuse; corrections go through site 6 per invoice, or the next run.

8 BatchCreatePayrollInvoices → DeletePayrollInvoicesSafelyAsync DELETE, GUARDED

ENTRY	POST /invoices/batchCreatePayrollInvoices, the re-run branch
REMOVES	Same wipe as site 7, scoped to the employees in the request. The payment deletion is not scoped: re-running for a subset of employees deletes every staff-party payment for the month.
BECOMES	Same guard as site 7, plus scope the payment deletion to the run being replaced. Do not re-run a month after this guard trips; the September report's warning about June stands.

Ledger, direct

9 DeleteAcctgTrans REVERSE DELETE WHILE UNPOSTED

ENTRY	DELETE /transactions/deleteAcctgTrans/{id} · Transactions list
REFUSES TODAY	Nothing. The posted guard exists in the source and is commented out. Any of the 5,194 posted transactions can be removed from this screen.
BECOMES	Posted → this endpoint becomes "Reverse", the natural home of the reversal service. Unposted (19 rows today) → delete. Journal-entry users are the audience the auditor is most concerned with.

10 DeleteAcctgTransEntry · UpdateMultiAcctgTransWithEntries EDIT ONLY UNPOSTED

ENTRY	DELETE /transactions/deleteAcctgTransEntry · PUT multi-entry editor
REMOVES	Individual entries from a header, in place, with no posted check.
BECOMES	Allowed only while <code>IS_POSTED = 'N'</code> ; posted → reverse and re-enter. Because <code>CreateAcctgTransAndEntries</code> stamps posted at creation, a true edit window for manual journals requires creating unposted and posting explicitly. That is a design decision for step 2, not a guard.

11 RecordInstallmentPaymentAccounting RETIRE

ENTRY	None. No controller, handler or frontend call references the command.
REMOVES	Deletes the existing <code>INCOMING_PAYMENT</code> transaction for a payment and posts a fresh one.
BECOMES	Dead code; remove it so it cannot be wired back in. If revived, it is a reverse-and-repost.

Construction certificates

12 ResetProjectCertificate **REVERSE**

ENTRY	POST /project/resetCertificate/{id}
REFUSES TODAY	Only a certificate already in WEPR_CREATED . A 300-second command timeout is set, which says how much it removes.
REMOVES	The order's payments with their transactions, applications and preferences; linked certificates; shipment receipts, inventory item details and their transactions; the invoices it generated, with their transactions and applications; shipments and their transactions. Then the header goes back to created.
EXPOSURE	497 approved project certificates; 1,767 work-effort-anchored and 242 shipment-or-receipt-anchored posted transactions.
BECOMES	Refuse while any payment on the certificate is beyond draft; those are voided individually first (site 2). Then reverse the approval's transaction set and set WEPR_CREATED . Generated invoices become INVOICE_CANCELLED with a reversal rather than disappearing.

13 DeleteProjectCertificate **VOID** **DELETE WHILE CREATED**

ENTRY	DELETE /project/deleteProjectCertificate/{id}
REFUSES TODAY	Nothing. Deletes the header, its items, the purchase order and all order dependents including payments and payment preferences. It does not remove transactions, so on an approved certificate it fails on a foreign key rather than by design.
BECOMES	WEPR_CREATED with no transactions (1 row today) → delete. Approved → WEPR_CANCELLED plus reversal.

14 UpdateProjectCertificate **EDIT ONLY CREATED**

ENTRY	PUT /project/updateProjectCertificate/{id}
REMOVES	On every edit, deletes the old purchase order and its dependents, payments and preferences included, and recreates them. No approval-status check.
BECOMES	Allowed only in WEPR_CREATED . An approved certificate is reset first (site 12), then edited.

15 ResetMultiPaymentCertificate **REVERSE**

ENTRY	POST /project/resetMultiPaymentCertificate/{id}
REFUSES TODAY	Anything but WEPR_APPROVED , which is the right precondition.
REMOVES	Every transaction anchored to the work effort, then sets header and items to created.
EXPOSURE	188 approved multi-payment certificates.
BECOMES	Reverse instead of remove. Otherwise unchanged; this is the smallest conversion in the register.

16 DeleteMultiPaymentCertificate **VOID** **DELETE WHILE CREATED**

ENTRY	DELETE /project/multiPaymentCertificate/{id}
REFUSES TODAY	Nothing. Removes transactions, items and the header regardless of status.
BECOMES	Created with no transactions (2 rows today) → delete. Approved → WEPR_CANCELLED plus reversal.

Apartment sales

17 DeleteSalesRequest **VOID** **DELETE WHILE CREATED**

ENTRY	DELETE /salesrequests/{id}
REFUSES TODAY	Nothing.
REMOVES	Commission payments, disbursed ones included; commission records; every customer payment with its bank rows, applications, attributes, allocations and contents; every sales-request-anchored transaction, including reconciliation entries; the instalment schedule; the request itself. The apartment goes back to available.
EXPOSURE	137 sales requests, all approved; 952 posted transactions; 798 received customer payments system-wide.
BECOMES	Request → cancelled, each received payment voided with its reversal, commission payments voided, apartment released. A <code>SALES_REQUEST_CANCELLED</code> status does not exist yet; only created and approved are seeded. Delete only while created with no payment beyond draft.

18 ResetSalesRequest **REVERSE**

ENTRY	Sales request form, reset action
REFUSES TODAY	Anything but approved; any commission approved or paid. Good preconditions.
REMOVES	Every payment and every anchored transaction through the shared purge helper, then status back to created and the apartment to reserved.
BECOMES	Also refuse while any customer payment has been received. Then reverse the approval postings only and return to created.

19 DeleteSalesCommission · ResetSalesCommission → CommissionPaymentCleanup

VOID **DELETE WHILE DRAFT**

ENTRY	Commission screen, delete and reset actions
REMOVES	Every <code>COMMISSION_PAYMENT</code> on the sales request through the shared purge helper, regardless of whether it was disbursed. Reset is limited to approved commissions; delete purges whenever the commission is approved.
EXPOSURE	184 disbursed commission payments; 22 approved commissions.
BECOMES	Draft commission payments → delete. Disbursed → void through the payment primitive. This is the same gap the withholding-tax follow-up already recorded.

5. Defects surfaced on the way

These are not part of the auditor's finding but were confirmed while reading the sites, and two of them change how step 2 is built.

1. **Status id mismatch.** `ResetPayment` guards on `PMNT_VOIDED`; the seeded status and the void routine both use `PMNT_VOID`. One of them is wrong, and today the guard never fires. Settle on `PMNT_VOID`, which is the OFBiz id.
2. **Bank rows on draft payments are by design, not a leak.** 90 fin-account rows sit on payments in `PMNT_NOT_PAID`. Checked on 14 September: the bank row is created together with a cheque payment at draft time (88 of the 206 draft cheque payments

carry one), so Reset must leave it alone and only Void cancels it. The earlier reading of these rows as a ResetPayment leak was wrong and no cleanup is needed.

3. **The commented-out guard.** `DeleteAcctgTrans` carries the posted check as a comment. Whoever removed it made the decision the auditor is now reversing.
4. **No orphans, by luck.** Zero transactions point at a missing payment or invoice. Foreign keys, not the handlers, are what has kept the ledger internally consistent.
5. **Nothing has ever been closed.** Six time periods, none closed, unchanged since the September report.

6. What this sizes for step 2

The classification reduces to five building blocks. Every site in the register is one of these applied to one artifact type, which is why the primitives come before any handler is touched.

Primitive	Used by sites	Starting point in the code	Gap to close
ReverseAcctgTrans: linked contra transaction	1, 4, 6, 9, 12, 15, 18 and every void	<code>GeneralLedgerService.CopyAcctgTransAndEntries(id, revert: true)</code> already clones a transaction with debit and credit swapped.	No link back to the original (copies <code>TheirAcctgTransId</code> verbatim). Date is "now" with no period check. Description not marked as a reversal. Never posted through a guard.
VoidPayment as a command and endpoint	2, 3, 13, 16, 17, 19	<code>PaymentHelperService.VoidPayment</code> : sets status, removes applications, returns paid invoices to ready, reverses transactions.	Only reachable from the cheque-run cancel path. Ignores fin-account rows. Only reverses transactions with no invoice id. Uses the reversal above, so inherits its gaps.
HasLedgerHistory refusal	5, 7, 8, and the draft branch of every void	<code>DeleteParty</code> 's check for <code>AcctgTrans</code> rows.	Extract to one query covering transactions, entries and fin-account rows, and call it from every physical delete.
Period guard that throws	Every site	The warning inside <code>PostAcctgTrans</code> .	Must run on create, reverse and delete paths, and must block, not warn.
Cancelled statuses	13, 16, 17, 19	<code>WEPR_CANCELLED</code> , <code>ADVANCE_CANCELLED</code> , <code>INVOICE_CANCELLED</code> , <code>PMNT_VOID</code> are seeded.	<code>SALES_REQUEST_CANCELLED</code> is missing. List queries need to exclude cancelled rows where they feed operations.

The one decision needed before building. Whether the pragmatic exception stands: within an open period, before any payment is sent, may an invoice accrual be unposted, edited and re-posted rather than reversed? It affects sites 4, 6, 7 and 8 and nothing else. The September report recommended pragmatic before send, strict after, and asked for the auditor's written agreement. That agreement is the input to step 2.

7. How this was verified

Every handler in the register was read in full at `master @ eeb49e0e` , not sampled. The sweep that extended the list from 13 to 19 searched the Application layer for any `ExecuteDeleteAsync` , `RemoveRange` or `Remove` applied to the four financial tables, including the multi-line form where the table and the delete sit on different lines. Entry

points were traced to their controller routes; frontend gating was checked for the payment reset. Counts were read from the local database with the queries below.

```
-- posted transactions by anchor
SELECT COUNT(*), SUM(IS_POSTED='Y') FROM ACCTG_TRANS WHERE PAYMENT_ID IS NOT NULL;
-- 3351 / 3351
SELECT COUNT(*), SUM(IS_POSTED='Y') FROM ACCTG_TRANS WHERE WORK_EFFORT_ID IS NOT NULL;
-- 1767 / 1767
SELECT COUNT(*), SUM(IS_POSTED='Y') FROM ACCTG_TRANS WHERE SALES_REQUEST_ID IS NOT
NULL;-- 952 / 952
SELECT COUNT(*), SUM(IS_POSTED='Y') FROM ACCTG_TRANS WHERE INVOICE_ID IS NOT NULL;
-- 550 / 550

-- the reset-payment leak
SELECT COUNT(*) FROM FIN_ACCOUNT_TRANS f JOIN PAYMENT p USING (PAYMENT_ID)
WHERE p.STATUS_ID = 'PMNT_NOT_PAID';
-- 90

-- seeded void status (not PMNT_VOIDED)
SELECT STATUS_ID FROM STATUS_ITEM WHERE STATUS_TYPE_ID='PMNT_STATUS' AND STATUS_ID LIKE
'%VOID%'; -- PMNT_VOID
```

Prepared 13 September 2026. Follows Part II of "Payroll: Status and Way Forward" (6 September 2026), which this document's numbering of steps refers to. Sections 4 to 6 are findings and recommendations, not committed work.